

PERTEMUAN VII

IMBALANCED DATASET



7.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep imbalanced dataset serta permasalahan yang muncul akibat ketidakseimbangan kelas dalam model machine learning
- B. Mahasiswa mampu menerapkan teknik balancing data seperti SMOTE dan ADASYN untuk menyeimbangkan distribusi kelas pada dataset
- C. Mahasiswa memahami konsep dan metode feature selection (filter, wrapper, embedded) untuk memilih fitur yang paling relevan

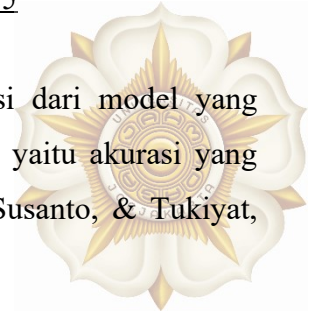
7.2 DASAR TEORI

A. Imbalanced Class

Imbalance class adalah kondisi di mana suatu himpunan data terdapat satu kelas yang memiliki jumlah yang jauh lebih kecil dibanding kelas lainnya. *Imbalance class* menyebabkan kelas minoritas diprediksi secara salah menjadi kelas mayoritas. Selain itu, *imbalance class* mengakibatkan tingkat sensitivitas rendah pada data dengan dua kategori atau binomial. Masalah lain yang dapat muncul yaitu banyaknya data kosong yang dapat menyebabkan beberapa algoritma dalam metode klasifikasi tidak dapat dijalankan. Hal ini mengakibatkan tingkat akurasi rendah dan kualitas data yang buruk saat data diolah. Data dengan *imbalance class* dan banyaknya *missing value* perlu diberikan perlakuan tertentu agar diperoleh metode klasifikasi dengan ukuran ketepatan prediksi yang tinggi. *Imbalance class* dapat diatasi dengan memanipulasi data, memodifikasi algoritma, atau gabungan keduanya (Istiana & Mustafiril, 2023).

B. Confusion Matrix

Confusion Matrix adalah suatu metode yang digunakan untuk melakukan perhitungan akurasi pada konsep data *mining* (Normawati & Prayogi, 2021). Matriks ini diimplementasikan dengan sebuah tabel yang digunakan untuk mengevaluasi kinerja algoritma klasifikasi dalam *machine learning*. Tabel ini membandingkan hasil prediksi model dengan label sebenarnya dari data, sehingga kita bisa melihat sejauh mana model melakukan prediksi dengan benar dan salah. Evaluasi performansi dilakukan untuk



menguji hasil dari klasifikasi dengan mengukur nilai performansi dari model yang telah dibuat. Parameter pengujian yang digunakan untuk evaluasi yaitu akurasi yang perhitungannya diperoleh dari tabel *confusion matrix* (Septiana, Susanto, & Tukiyyat, 2021).

C. Feature Selection

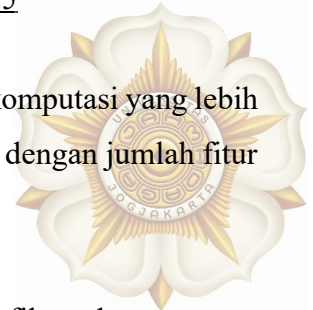
Feature selection merupakan sebuah proses identifikasi subset fitur yang paling relevan pada kumpulan fitur yang berguna untuk meningkatkan kinerja algoritma (Pratama & Kusnawi, 2023). Salah satu faktor penting untuk meningkatkan nilai akurasi klasifikasi adalah seleksi fitur. Apabila dataset berisi fitur yang banyak, maka dimensi ruang menjadi besardantingkat akurasi klasifikasi menjadi rendah. Seleksi fitur digunakan untuk mengurangi jumlah set fitur yang besar menjadi subset fitur yang relatif lebih kecil agar nilai akurasi klasifikasi meningkat. Seleksi fitur diperlukan untuk mendapatkan fitur-fitur yang relevan yang selanjutnya akan digunakan dalam proses konstruksi model (Septiana, Susanto, & Tukiyyat, 2021).

D. Filter Method

Filter method merupakan pendekatan feature selection yang mengevaluasi relevansi setiap fitur berdasarkan sifat statistiknya terhadap variabel target tanpa melibatkan model pembelajaran (Guyon & Elisseeff, 2003). Metode ini umumnya menggunakan ukuran seperti korelasi antara fitur dan target untuk menilai kekuatan hubungan, di mana fitur dengan nilai korelasi lebih tinggi dianggap lebih informatif. Selain itu, uji statistik seperti ANOVA (Analysis of Variance) juga sering digunakan untuk mengukur sejauh mana suatu fitur mampu membedakan nilai mean antar-kelompok berdasarkan label, sehingga fitur dengan nilai F-test tinggi dinilai lebih relevan. Dengan sifatnya yang sederhana dan efisien, filter method menjadi langkah awal yang penting dalam pemilihan fitur sebelum menerapkan metode seleksi yang lebih kompleks.

E. Wrapper Method

Wrapper method merupakan pendekatan feature selection yang mengevaluasi subset fitur dengan menggunakan model prediktif secara langsung. Teknik ini mencoba berbagai kombinasi fitur—baik dengan forward selection, backward elimination, atau recursive feature elimination—dan memilih subset yang menghasilkan performa model terbaik. Karena melibatkan proses pelatihan model berulang kali, wrapper method lebih



akurat dibandingkan filter method, tetapi membutuhkan waktu dan komputasi yang lebih besar. Namun, metode ini tetap sangat efektif terutama pada dataset dengan jumlah fitur yang tidak terlalu besar. (Kohavi & John, 1997).

F. Embedded Method

Embedded method merupakan gabungan keunggulan dari filter dan wrapper method. Pada pendekatan ini, proses pemilihan fitur dilakukan bersamaan dengan proses pelatihan model, sehingga lebih efisien dibandingkan wrapper namun lebih akurat dibandingkan filter. Contoh metode embedded yang paling umum adalah LASSO (Least Absolute Shrinkage and Selection Operator) yang melakukan regularisasi L1 untuk mendorong beberapa koefisien fitur menjadi nol, serta Tree-based models seperti Random Forest dan Gradient Boosting yang menggunakan feature importance dalam proses pemilihan fitur. Embedded method menjadi favorit karena keseimbangan antara akurasi dan efisiensi komputasi. (Tibshirani, 1996).

G. SMOTE

SMOTE merupakan metode *oversampling* yang paling populer digunakan. SMOTE dilakukan dengan menambah data sintesis pada kelas minoritas. Data sintesis merupakan data baru yang dibangkitkan. SMOTE dilakukan dengan menambah jumlah data pada kelas minoritas dengan cara membangkitkan data baru berdasarkan K tetangga terdekat. Data pada kelas minoritas dilakukan *oversampling* dengan mengambil data pada kelas minoritas dan menambah sampel sintesis di sepanjang garis yang menghubungkan salah satu atau semua K tetangga terdekat data kelas minoritas tersebut. Jumlah tetangga K dipilih secara acak (Wijayanti, Kencana, & Sumarjaya, 2021).

H. ADASYN

Adaptive Synthetic Sampling Approach (ADASYN) merupakan teknik untuk mengatasi masalah data *imbalance* dengan melakukan *oversampling* pada kelas minoritas yaitu dengan menggunakan bobot distribusi untuk data pada kelas minoritas berdasarkan pada tingkat kesulitan pembelajaran model. Data sintesis dihasilkan dari data minoritas yang sulit untuk dipahami dibandingkan dengan data minoritas yang lebih mudah untuk dipahami (Ramadhanti, Santoso, & Widiari, 2022).



7.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

7.4 LANGKAH PERCOBAAN

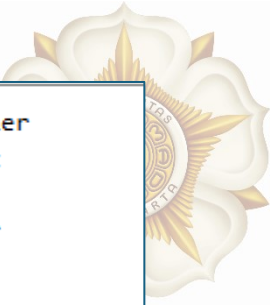
1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Google Colab dipilih karena telah mendukung berbagai library machine learning yang dibutuhkan, seperti pandas, numpy, matplotlib, seaborn, dan scikit-learn. Selain itu, Colab menyediakan resource komputasi berbasis CPU atau GPU secara gratis sehingga proses pengolahan data, pemodelan, serta penerapan teknik feature selection dapat berjalan dengan lebih efisien tanpa membebani perangkat mahasiswa.

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor seluruh library yang dibutuhkan untuk melakukan preprocessing, visualisasi, dan proses feature selection, meliputi:

1. Pandas dan Numpy untuk manipulasi data
2. Matplotlib dan Seaborn untuk visualisasi, serta scikit-learn yang menyediakan berbagai fungsi untuk pemilihan fitur seperti SelectKBest, RFE, dan SelectFromModel.
3. Scikit-learn juga digunakan untuk membangun model klasifikasi (seperti Decision Tree dan SVM) serta evaluasi performa menggunakan metrik seperti accuracy, precision, recall, dan confusion matrix.



```

from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import RandomOverSampler
from scipy.stats import norm
from imblearn.over_sampling import ADASYN
from imblearn.over_sampling import SMOTE

import seaborn as sns
import matplotlib.pyplot as plt
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

Gambar 7.4.1 Impor Pustaka

3. Import dan Load Dataset Information

Mahasiswa kemudian memuat Stroke Prediction Dataset ke dalam DataFrame. Dataset ini berisi informasi klinis dan demografis pasien yang dapat digunakan untuk memprediksi potensi terjadinya stroke. Fitur-fitur di dalam dataset mencakup usia, jenis kelamin, riwayat hipertensi, riwayat penyakit jantung, status merokok, indeks massa tubuh (BMI), serta nilai glukosa darah. Pada tahap ini mahasiswa melakukan beberapa langkah penting, yaitu:

1. Mengimpor dataset dari sumber online atau Google Drive menuju DataFrame pandas.
2. Menampilkan beberapa baris awal dataset untuk memastikan data berhasil dimuat dan tidak terjadi kesalahan pembacaan.

```

# mount drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

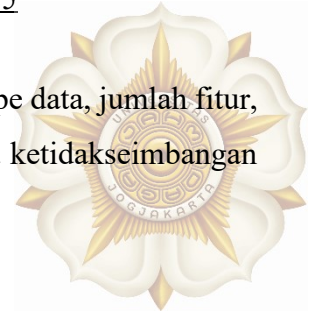
# read file
df = pd.read_csv('/content/drive/MyDrive/P Penambangan Data Sem 6/healthcare-dataset-stroke-data-latihan.csv')

# baris data dari atas
df.head()

```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 7.4.2 Pembacaan Dataset



3. Menganalisis struktur dataset menggunakan untuk memahami tipe data, jumlah fitur, nilai statistik dasar, serta memeriksa adanya missing value atau ketidakseimbangan kelas pada target (stroke).

```
# cek missing values
df.isnull().sum()

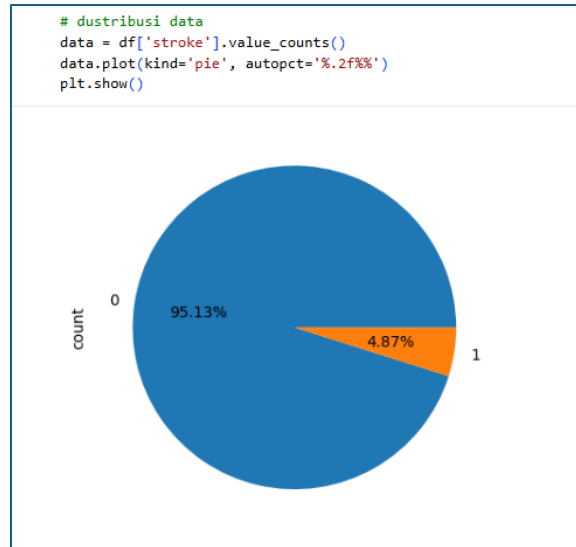
***
0
id      0
gender  0
age     0
hypertension  0
heart_disease  0
ever_married  0
work_type  0
Residence_type  0
avg_glucose_level  0
bmi      201
smoking_status  0
stroke   0

dtype: int64
```

Gambar 7.4.3 Mengecek Null Value

4. Exploratory Data Analysis

Mahasiswa memvisualisasikan distribusi variabel target (stroke) menggunakan grafik pie untuk mengetahui apakah dataset memiliki ketidakseimbangan kelas. Visualisasi ini penting untuk menentukan apakah teknik balancing seperti SMOTE atau ADASYN perlu digunakan pada tahap selanjutnya.

*Gambar 7.4.4 Distribusi data*

5. Data Cleaning

Pada tahap ini dilakukan pemeriksaan kualitas data berdasarkan hasil `isnull()`. Pada fitur `bmi` ditemukan 201 missing value. Karena BMI merupakan variabel numerik dan memiliki variasi yang cukup stabil, missing value diatasi menggunakan teknik imputasi dengan mengisi nilai kosong menggunakan nilai rata-rata (mean) dari fitur tersebut.

```
# imputasi missing value

df['bmi'].fillna(df['bmi'].mean(), inplace=True)
```

Gambar 7.4.5 Data Cleaning



```
# cek missing values after imputation
df.isnull().sum()

***
0
id      0
gender  0
age     0
hypertension  0
heart_disease  0
ever_married  0
work_type  0
Residence_type  0
avg_glucose_level  0
bmi      0
smoking_status  0
stroke   0

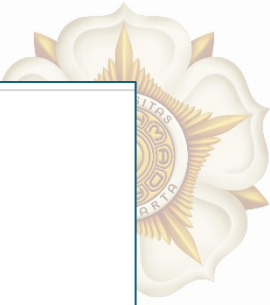
dtype: int64
```

Gambar 7.4.6 Mengecek jumlah data null

6. Random Under Sampling

Setelah data cleaning, dataset dipisahkan menjadi fitur (df_X) dan target (df_y). Kolom bmi yang memiliki nilai kosong diisi dengan rata-rata, dan variabel kategori seperti gender, ever_married, work_type, Residence_type, dan smoking_status dikodekan agar dapat digunakan dalam pemodelan. Semua fitur kemudian dikonversi ke tipe numerik.

Dataset dibagi menjadi data train dan test dengan rasio 75:25. Karena jumlah sampel kelas mayoritas jauh lebih banyak dibanding kelas minoritas, data train diseimbangkan menggunakan RandomUnderSampler, sehingga jumlah sampel pada kedua kelas menjadi sama. Distribusi kelas sebelum dan sesudah proses ini ditampilkan untuk memastikan kesetimbangan data.



```

from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

under_sampling = RandomUnderSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

le = LabelEncoder()
df_y = le.fit_transform(df_y)

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

X_resampled, y_resampled = under_sampling.fit_resample(X_train, y_train)

print('sebelum undersampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah undersampling')
print(pd.DataFrame(y_resampled).value_counts())

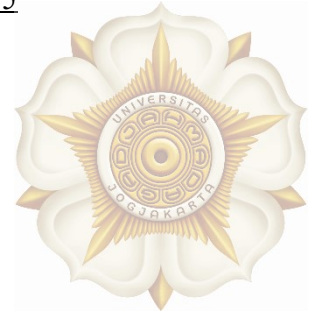
```

```

... sebelum undersampling
0
0    3663
1     169
Name: count, dtype: int64
setelah undersampling
0
0     169
1     169
Name: count, dtype: int64

```

Gambar 7.4.7 Random Sampling

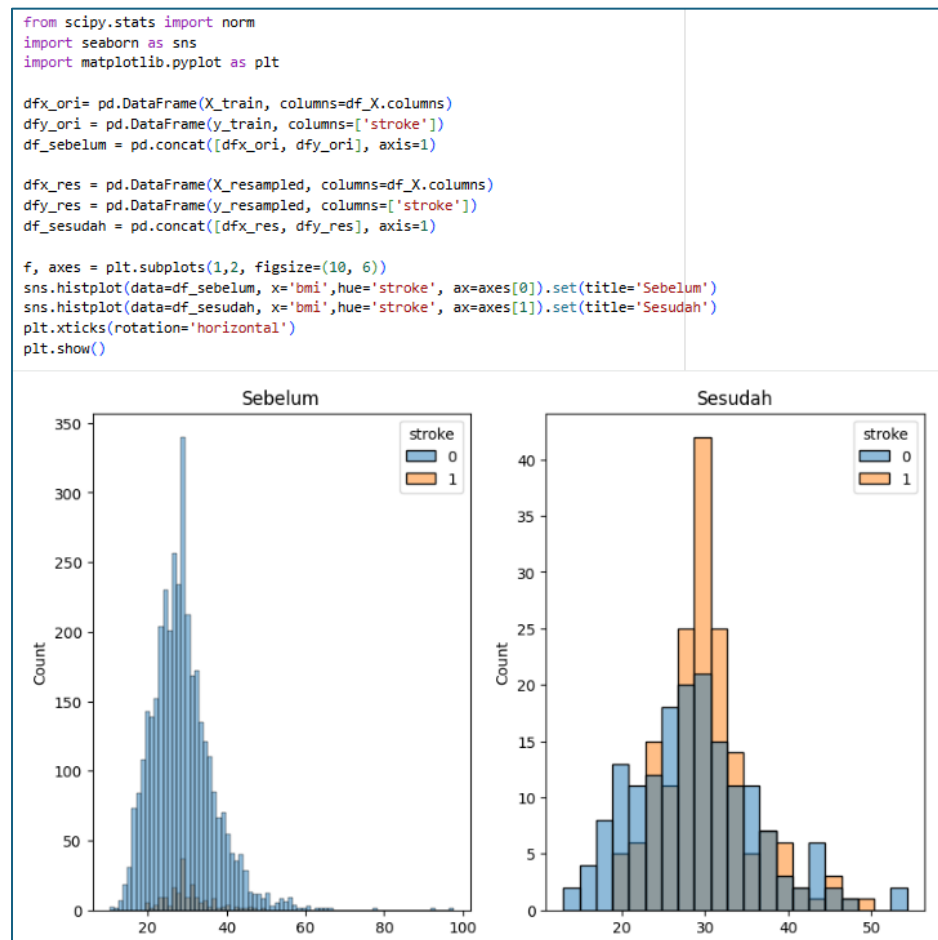


```
# menampilkan kolom stroke
df_y = df[['stroke']]
df_y
```

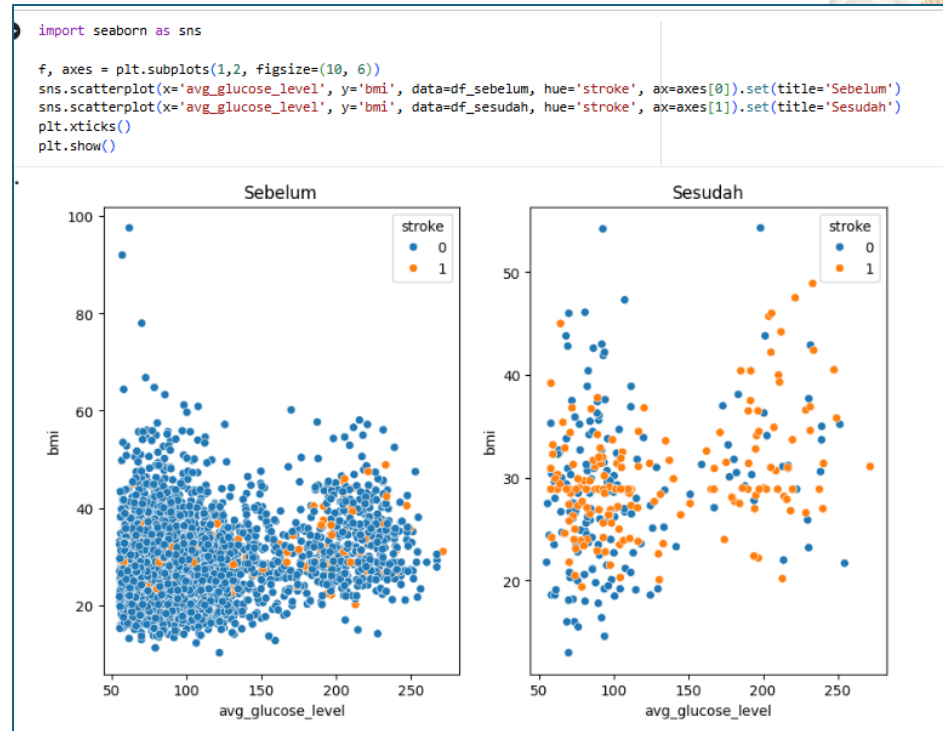
	stroke
0	1
1	1
2	1
3	1
4	1
...	...
5105	0
5106	0
5107	0
5108	0
5109	0

5110 rows × 1 columns

Gambar 7.4.8 Melihat data



Gambar 7.4.9 Perbandingan sebelum random sampling dan sesudah



Gambar 7.4.10 Visualisasi Scatter

7. Random Over Sampling

Selain undersampling, salah satu teknik untuk menangani imbalanced data adalah oversampling. Setelah data cleaning dan pemisahan fitur serta target, RandomOverSampler digunakan untuk menambah jumlah sampel pada kelas minoritas sehingga kedua kelas memiliki jumlah yang seimbang. Distribusi kelas sebelum dan sesudah oversampling ditampilkan untuk memastikan data train menjadi seimbang dan siap digunakan dalam pemodelan.



```
from imblearn.over_sampling import RandomOverSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

over_sampling = RandomOverSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values

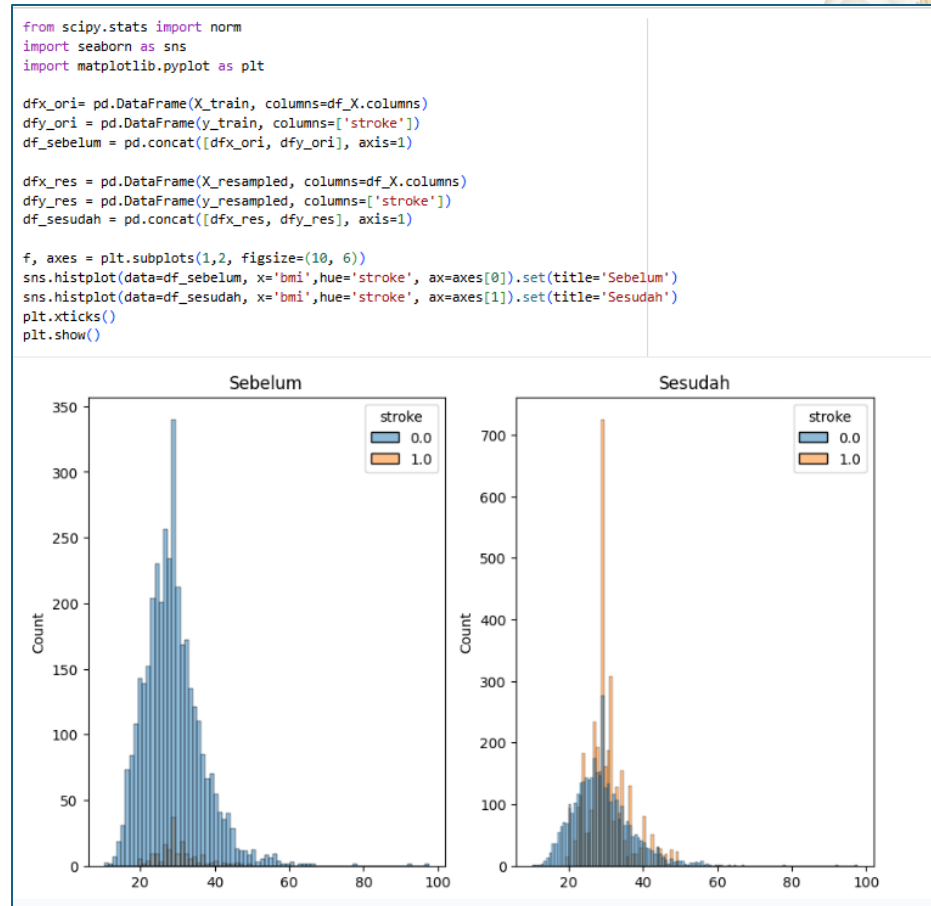
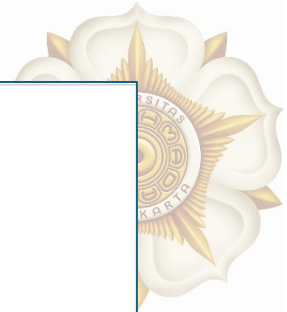
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

X_resampled, y_resampled = over_sampling.fit_resample(X_train, y_train)

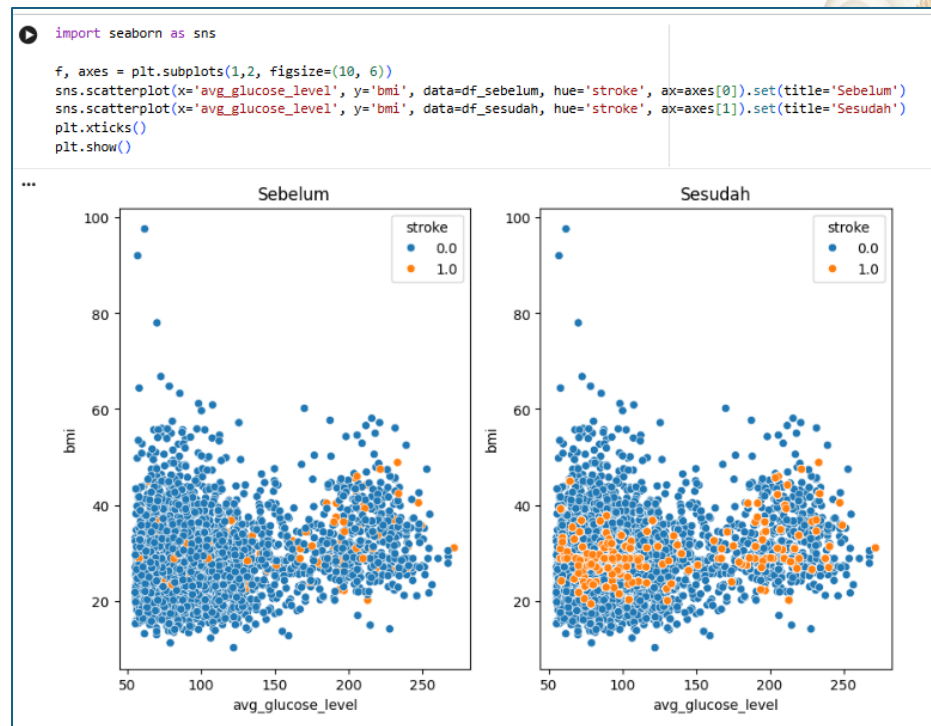
print('sebelum random oversampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah random oversampling')
print(pd.DataFrame(y_resampled).value_counts())
```

```
*** sebelum random oversampling
0
0.0    3663
1.0     169
Name: count, dtype: int64
setelah random oversampling
0
0.0    3663
1.0    3663
Name: count, dtype: int64
```

Gambar 7.4.11 Random over sampling



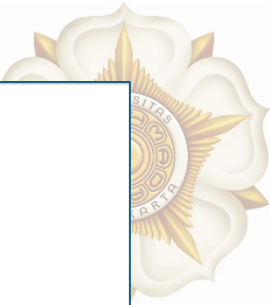
Gambar 7.4.12 Sebelum vs Sesudah over sampling



Gambar 7.4.13 Visualisasi scatter

8. SMOTE

Selain undersampling dan oversampling biasa, teknik lain untuk menangani imbalanced data adalah SMOTE (Synthetic Minority Over-sampling Technique). Setelah data cleaning dan pemisahan fitur serta target, SMOTE digunakan untuk membuat sampel sintetis pada kelas minoritas, sehingga jumlah sampel pada kedua kelas menjadi seimbang tanpa menghapus data kelas mayoritas. Distribusi kelas sebelum dan sesudah SMOTE ditampilkan untuk memastikan data train seimbang dan siap digunakan untuk pelatihan model.



```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

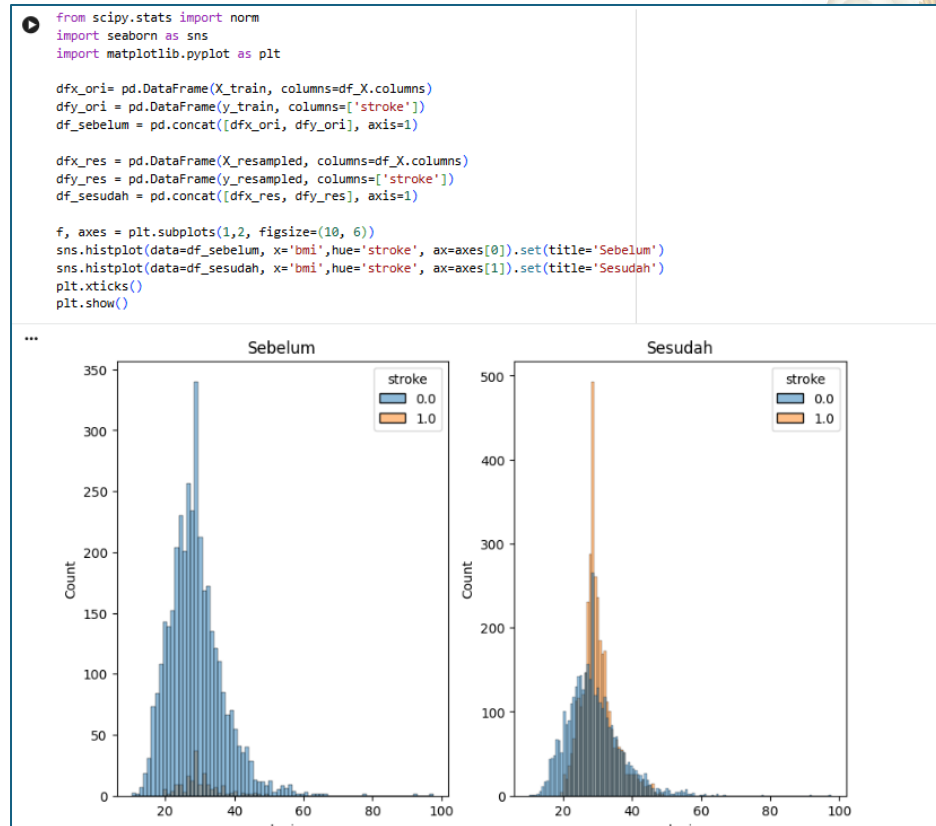
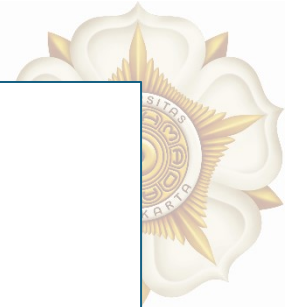
# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
smote = SMOTE(random_state=0)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

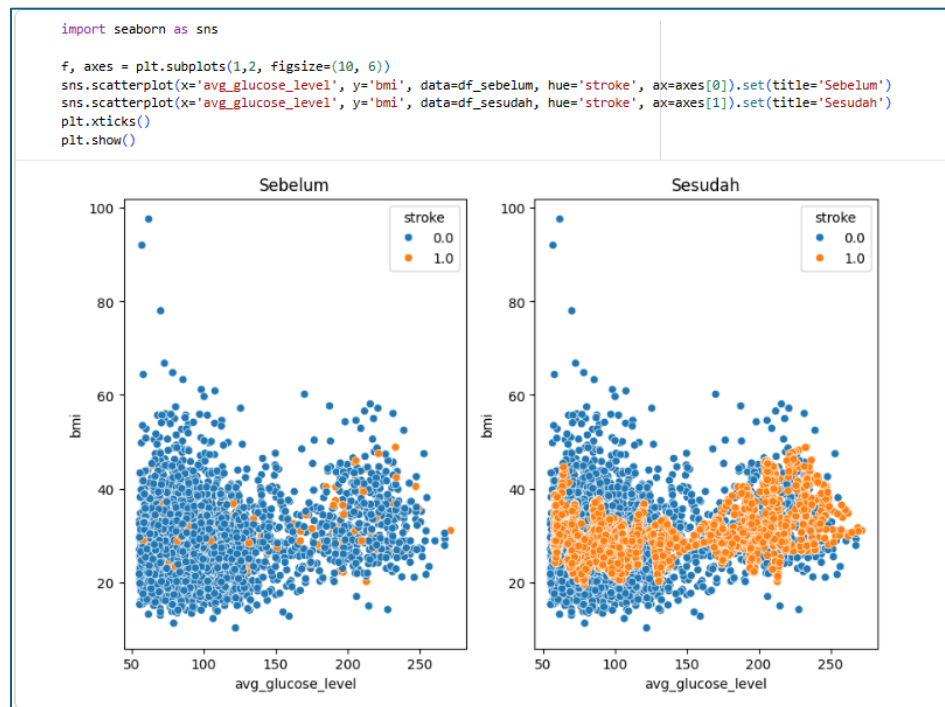
print('Sebelum SMOTE')
print(pd.DataFrame(y_train).value_counts())
print('Setelah SMOTE')
print(pd.DataFrame(y_resampled).value_counts())
```

```
*** Sebelum SMOTE
0
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah SMOTE
0
0.0    3663
1.0    3663
Name: count, dtype: int64
```

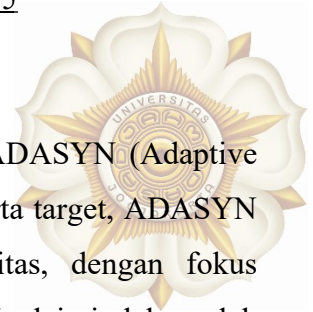
Gambar 7.4.14 SMOTE



Gambar 7.4.15 Sebelum vs Sesudah SMOTE



Gambar 7.4.16 Visualisasi Scatter



9. ADASYN

Selain SMOTE, salah satu teknik penyeimbangan kelas adalah ADASYN (Adaptive Synthetic Sampling). Setelah data cleaning dan pemisahan fitur serta target, ADASYN digunakan untuk membuat sampel sintetis pada kelas minoritas, dengan fokus menambahkan lebih banyak sampel di sekitar titik-titik yang sulit dipelajari oleh model. Teknik ini membantu model belajar lebih baik dari daerah kelas minoritas yang kompleks. Distribusi kelas sebelum dan sesudah ADASYN ditampilkan untuk memastikan data train seimbang dan siap digunakan untuk pelatihan model.

```

from imblearn.over_sampling import ADASYN
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender'] = le.fit_transform(df_X['gender'])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type'] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status'] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
adasyn = ADASYN(random_state=0)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)

print('Sebelum ADASYN')
print(pd.DataFrame(y_train).value_counts())
print('Setelah ADASYN')
print(pd.DataFrame(y_resampled).value_counts())

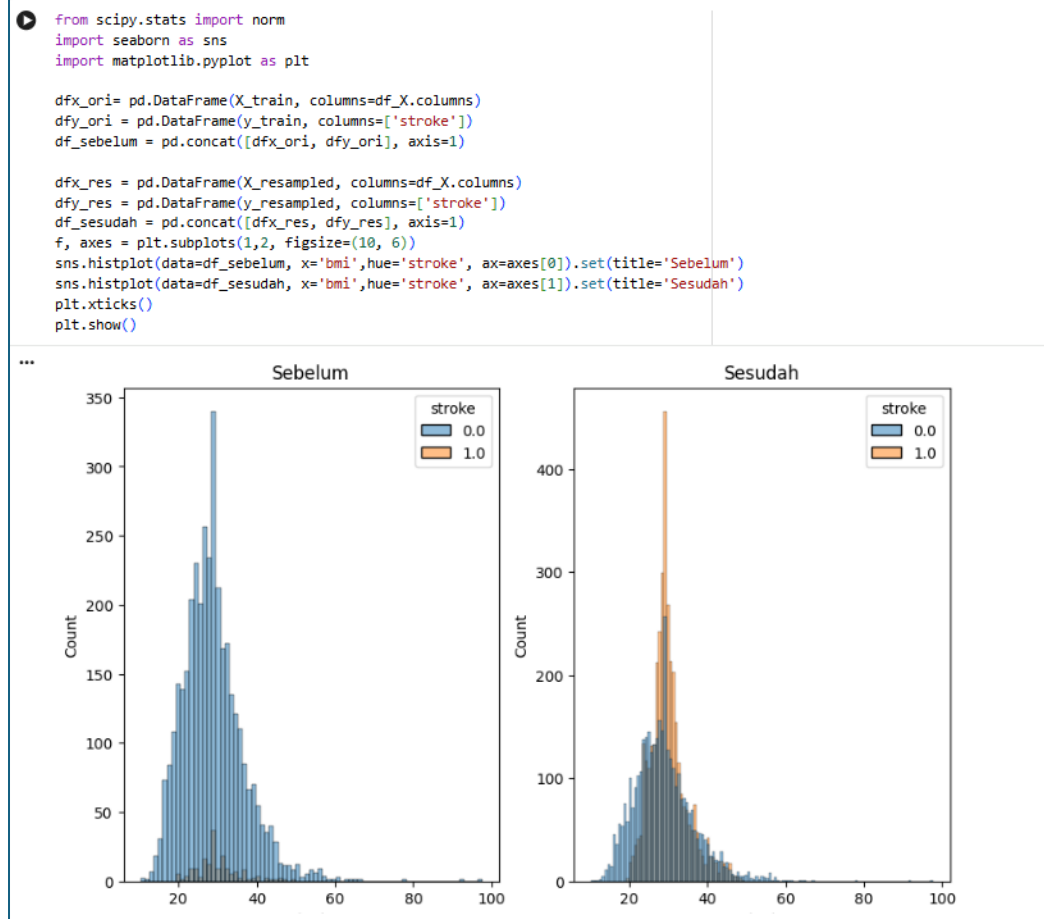
```

```

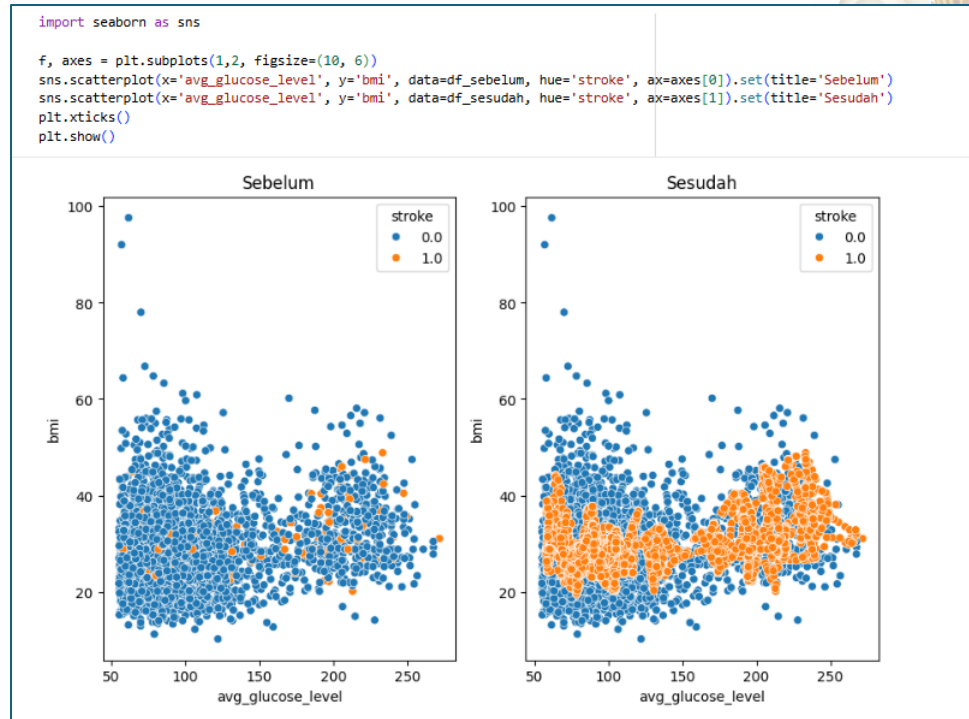
... Sebelum ADASYN
0
0.0 3663
1.0 169
Name: count, dtype: int64
Setelah ADASYN
0
1.0 3676
0.0 3663
Name: count, dtype: int64

```

Gambar 7.4.17 ADASYN



Gambar 7.4.18 Sebelum dan Sesudah Adasyn



Gambar 7.4.19 Visualisasi scatter

7.5 TUGAS DAN ANALISIS

1. Tugas 1

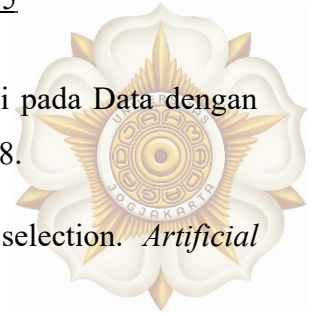
- Download dataset <https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>
- Upload dataset ke gdrive atau github masing-masing
- Tampilkan dataset dalam dataframe
- Jelaskan tujuan penggunaan dataset ini
- Jelaskan feature dari dataset ini

2. Tugas 2

- Dengan menggunakan dataset lain, kombinasikan SMOTE atau ADASYN dengan classifier.
- Tampilkan performa model (accuracy, precision, recall) sebelum dan sesudah data balancing pada training set nya

7.6 REFERENSI

Machine Learning Research, 3, 1157–1182.



Istiana, A., & Mustafiril, M. (2023). Perbandingan Metode Klasifikasi pada Data dengan Imbalance Class dan Missing Value. *JURNAL INFORMATIKA*, 101-108.

Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.

Normawati, & Prayogi, T. (2021). Implementasi Naïve Bayes Classifier Dan Confusion Matrix Pada Analisis Sentimen Berbasis Teks Pada Twitter. *J-SAKTI (Jurnal Sains Komputer & Informatika)*, 697-711.

Pratama, R., & Kusnawi, K. (2023). Komparasi Algoritma Supervised Learning dan Feature Selection pada Klasifikasi Penyakit Gagal Jantung. *Indonesian Journal of Computer Science*, 3722-3733.

Ramadhanti, A. P., Santoso, H. B., & Widiharhi, T. (2022). PERBANDINGAN SMOTE DAN ADASYN PADA DATA IMBALANCE UNTUK KLASIFIKASI RUMAH TANGGA MISKIN KABUPATEN TEMANGGUNG DENGAN ALGORITMA K-NEAREST NEIGHBOR. *JURNAL GAUSSIAN*, 499 - 505.

Septiana, R., Susanto, A., & Tukiya. (2021). Analisis Sentimen Vaksinasi Covid-19 Pada Twitter Menggunakan Naive Bayes Classifier Dengan Feature Selection Chi-Squared Statistic Dan Particle Swarm Optimization. *Jurnal Sistem Komputer dan Kecerdasan Buatan*, 49-56.

Tibshirani, R. (1996). Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.

Wijayanti, N. L. P., Kencana, I. W., & Sumarjaya, I. M. (2021). SMOTE: POTENSI DAN KEKURANGANNYA PADA SURVEI. *E-Jurnal Matematika*, 235-240.